

PROGRAMACIÓN AVANZADA

Sesión 22

Archivos binarios

→ NOTAS DE CLASE

Índice

1	1973: los bytes que se reconocen solos	2
2	El problema del modo texto	3
3	El modo binario: write y read	4
3.1	Desglose de write y read	4
3.2	El cast (char*) &x	5
4	Guardar structs completos	5
4.1	Escribir un struct	6
4.2	Leer un struct	6
4.3	Múltiples registros y acceso aleatorio	7
5	Formatos binarios en el mundo real	7
5.1	BMP: el formato de imagen más simple	7
5.2	JPEG: compresión con pérdida	8
5.3	WAV: audio sin compresión	9
5.4	ELF: el ejecutable que ya conoces	9
6	Actividad: agenda de cumpleaños binaria	9
6.1	Formato del archivo	10
6.2	Parte A: genera tu archivo (10 min)	10
6.3	Parte B: las consultas (40 min)	10
7	Ejercicios	10

1973: los bytes que se reconocen solos

En la sesión anterior viste que un archivo no es más que una secuencia de bytes almacenada en disco. Esa definición aplica tanto a un `.txt` como a un `.jpg`, un `.mp3` o el ejecutable `a.out` que produces cada vez que compilas tu programa. La diferencia no está en el soporte físico sino en *cómo se interpretan* esos bytes.

En 1973, los laboratorios Bell publicaron la Cuarta Edición de Unix. Entre las herramientas que incluyó había una pequeña pero ingeniosa: el comando `file`. Su función era leer los primeros bytes de un archivo y determinar de qué tipo era, sin depender del nombre ni de la extensión. La herramienta era posible porque los diseñadores de formatos habían adoptado una convención: los primeros bytes de cada formato forman una **firma** (o **número mágico**) que lo identifica de forma única.

Hoy esa convención es universal. Cuando abres una foto, escuchas una canción o ejecutas un programa, el sistema operativo lee los primeros bytes del archivo para saber cómo procesarlo.

Formato	Firma (hex)	ASCII	Uso
ELF	7F 45 4C 46	\x7FELF	Ejecutables Linux
BMP	42 4D	BM	Imágenes (Windows)
JPEG	FF D8 FF	—	Imágenes
PNG	89 50 4E 47	\x89PNG	Imágenes
MP3 (ID3)	49 44 33	ID3	Audio
WAV	52 49 46 46	RIFF	Audio
ZIP	50 4B 03 04	PK	Archivos comprimidos
PDF	25 50 44 46	%PDF	Documentos

Aquí viene la sorpresa: el archivo `a.out` que produces cuando compilas con `g++` es un archivo binario en formato **ELF** (*Executable and Linkable Format*). Sus primeros cuatro bytes son siempre `7F 45 4C 46`. Puedes comprobarlo en tu terminal:

```
$ g++ hola.cpp -o hola
$ file hola
hola: ELF 64-bit LSB pie executable, x86-64, ...
$ xxd hola | head -1
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF.....
```

Llevas un semestre produciendo archivos binarios sin saberlo. Hoy vas a entender cómo funcionan y cómo crearlos tú mismo.

El término “número mágico” viene del código fuente original de Unix. Los primeros dos bytes de los ejecutables PDP-11 eran el valor octal `0407`, que el *loader* buscaba antes de ejecutar el archivo. Hoy la base de datos de firmas conocidas (`/usr/share/misc/magic`) contiene miles de entradas, desde formatos de los años 70 hasta los más recientes.

Referencia: Ritchie, D. M. (1984). “The Evolution of the Unix Time-sharing System.” *AT&T Bell Laboratories Technical Journal*, 63(8), 1577–1593.

El problema del modo texto

En la sesión anterior guardaste enteros en archivos de texto con el operador `<<`. Funciona bien para mostrar resultados, pero tiene un problema cuando los datos deben ser leídos por otro programa.

Considera el siguiente código:

binarios1.cpp:

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){
    int x = 123;
    int y = 27;

    ofstream f1("datos.out");
    f1 << x << y;
    f1.close();

    ifstream f2("datos.out");
    int z;
    f2 >> z;
    cout << z << endl;
}
```

¿Qué imprime? Antes de leer la respuesta, razona: ¿qué contiene `datos.out` después de la escritura?

El operador `<<` convierte los enteros a su representación en texto. 123 se convierte en los tres caracteres `'1'`, `'2'`, `'3'`, y 27 en los dos caracteres `'2'`, `'7'`. El archivo queda así:

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4
0x31	0x32	0x33	0x32	0x37
'1'	'2'	'3'	'2'	'7'

Cinco bytes que forman la cadena "12327". Cuando `f2 >> z` intenta leer un entero, lee todos los dígitos hasta el final del archivo y obtiene 12327. El programa imprime **12327**, no **123**.

La solución parcial en modo texto es agregar un separador:

```
f1 << x << " " << y;
```

Pero esto tiene dos problemas:

1. Hay que recordar poner el separador en la escritura y manejarlo en la lectura. Si los datos son más complejos (cadenas con espacios, tipos mixtos), el parseo se vuelve frágil.

2. Es ineficiente: el número 2147483647 (el mayor `int`) ocupa 10 bytes como texto, pero en binario siempre ocupa 4.

El **modo binario** resuelve ambos problemas de raíz.

✓ Recuerda

“Binario” no significa que el archivo contenga ceros y unos escritos como texto. Todos los archivos son binarios a nivel del disco. “Modo binario” significa que no hay conversión entre el dato en memoria y los bytes en disco: se copian los bytes exactos tal como están en la RAM.

El modo binario: write y read

Para abrir un archivo en modo binario, agrega la bandera `fstream::binary` al constructor. Luego, en lugar de `<<` y `>>`, se usan los métodos `write` y `read`:

binarios2.cpp:

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){
    int x = 123;
    int y = 27;

    ofstream f1("datosbin.out", fstream::binary);
    f1.write( (char *) &x, sizeof(int) );
    f1.write( (char *) &y, sizeof(int) );
    f1.close();

    ifstream f2("datosbin.out", fstream::binary);
    int w, z;
    f2.read( (char *) &w, sizeof(int) );
    f2.read( (char *) &z, sizeof(int) );
    cout << w << " " << z << endl;
}
```

Salida:

123 27

El archivo `datosbin.out` tiene exactamente **8 bytes**: 4 para `x` y 4 para `y`, copiados directamente de la memoria.

• Desglose de write y read

Ambos métodos tienen la misma firma:

```
ostream& write(const char* ptr, streamsize n);
istream& read(char* ptr, streamsize n);
```

- ptr** Puntero a los bytes que se van a escribir (o donde se van a colocar los bytes leídos). El tipo es `const char*`, que en C++ es el tipo básico de “puntero a bytes”.
- n** Número de bytes a escribir o leer. Casi siempre es `sizeof` del tipo que quieres guardar.

• El cast (`char*`) `&x`

Esta es la parte que más confunde. Analízala en dos partes:

1. `&x` es la dirección de memoria de la variable `x`. Su tipo es `int*` (“puntero a int”).
2. `(char *)` es un **cast**: le dices al compilador “trata esta dirección como un puntero a bytes, no como un puntero a int”.

No estás cambiando los datos. La variable `x` sigue siendo un `int` con el valor 123. Solo le estás diciendo a `write`: “empieza en esta dirección y copia los próximos `sizeof(int)` bytes tal como están”.

El cast `(char*)` es necesario porque `write` no sabe si los datos son `int`, `double`, un `struct` u otra cosa. Solo trabaja con bytes. El programador es responsable de indicar la dirección correcta y el tamaño correcto. Si el tamaño que pasas a `write` no coincide con el tamaño real del dato, los datos llegan corruptos al disco, y el error suele ser silencioso.

Después de ejecutar **binarios2.cpp** puedes confirmar que el archivo tiene exactamente 8 bytes:

```
$ ls -l datosbin.out
-rw-r--r-- 1 usuario grupo 8 may 11 datosbin.out
```

Compara con el archivo en modo texto: tendría 5 bytes ("12327") o 6 con el separador ("123 27"). El archivo binario es más pequeño y no necesita parseo.

Guardar structs completos

El verdadero poder del modo binario aparece cuando trabajas con **structs**. Un struct es un bloque contiguo de bytes en memoria. `write` puede copiar ese bloque completo al disco en una sola llamada, sin necesidad de escribir campo por campo.

Reutilizamos el struct de la actividad anterior:

persona.h:

```
using namespace std;
```

```
struct Persona {
```

```
    char nombre[30];
    char apellido[30];
    int dia;
    int mes;
};
```

• Escribir un struct

```
#include <iostream>
#include <fstream>
#include <cstring>
#include "persona.h"
using namespace std;

int main(){
    Persona p;
    strcpy(p.nombre, "Hannah");
    strcpy(p.apellido, "Fabián");
    p.dia = 29;
    p.mes = 2;

    ofstream f("hannah.cumple", fstream::binary);
    f.write( (char *) &p, sizeof(Persona) );
    f.close();
}
```

`sizeof(Persona)` calcula el tamaño del struct en bytes: en principio $30 + 30 + 4 + 4 = 68$, aunque el compilador puede agregar relleno (*padding*); ver el ejercicio 3.

• Leer un struct

```
#include <iostream>
#include <fstream>
#include "persona.h"
using namespace std;

int main(){
    Persona p;

    ifstream f("hannah.cumple", fstream::binary);
    f.read( (char *) &p, sizeof(Persona) );
    f.close();

    cout << p.nombre << " " << p.apellido
         << ": " << p.dia << "/" << p.mes << endl;
}
```

Salida:

Hannah Fabián: 29/2

La simetría es total: escritura y lectura usan `&p` y `sizeof(Persona)`. Si en alguno de los dos lados cambias el tamaño o la dirección, los datos llegan corruptos. Escribe primero, lee inmediatamente y verifica la salida antes de continuar.

• Múltiples registros y acceso aleatorio

Para guardar varios registros, llama a `write` una vez por registro:

```
ofstream f("agenda.bin", fstream::binary);
for (int i = 0; i < n; i++){
    f.write( (char *) &personas[i], sizeof(Persona) );
}
f.close();
```

El archivo resultante tiene exactamente `n * sizeof(Persona)` bytes. Para leer directamente el registro en la posición k (sin leer los anteriores), usa `seekg` para saltar:

```
ifstream f("agenda.bin", fstream::binary);
Persona p;
f.seekg(k * sizeof(Persona));
f.read( (char *) &p, sizeof(Persona) );
```

Esto se llama **acceso aleatorio**: puedes ir directo a cualquier registro como si fuera el elemento k de un arreglo. Con archivos de texto no es posible hacerlo de forma confiable porque los registros tienen longitud variable.

El struct `Persona` usa `char nombre[30]` en lugar de `string`. Un `string` en C++ no es un bloque contiguo de bytes: es un objeto que guarda internamente un puntero a memoria dinámica. Si lo escribieras en binario, guardarías la dirección del puntero, no el texto. En otra ejecución esa dirección no tendría sentido. Los arreglos de `char` son simples bloques de bytes y por eso funcionan correctamente con `write` y `read`.

Formatos binarios en el mundo real

Todos los formatos de archivo que usas a diario son binarios: imágenes, canciones, documentos, ejecutables. Cada uno tiene una estructura definida por un estándar y es, en esencia, un conjunto de structs escritos en disco. A continuación verás cómo están contruidos algunos de los más comunes.

• BMP: el formato de imagen más simple

El formato **BMP** (*BitMaP*) fue introducido con Windows 3.0 en 1990. No aplica compresión: los píxeles se almacenan directamente, lo que lo hace simple de leer pero voluminoso. Una imagen de 100×100 píxeles a 24 bits ocupa $100 \times 100 \times 3 = 30,000$ bytes de datos de imagen, más el encabezado.

La estructura es casi directamente expresable como structs de C++:


```
// Encabezado del archivo (14 bytes)
struct BMPHeader {
    char  firma[2];      // siempre "BM"
    int   tamano;        // tamaño total del archivo en bytes
    short reservado1;    // siempre 0
    short reservado2;    // siempre 0
    int   offset_datos;  // posición donde empiezan los píxeles
};

// Encabezado de imagen (40 bytes)
struct BMPInfo {
    int    tam_encabezado; // siempre 40
    int    ancho;          // píxeles de ancho
    int    alto;          // píxeles de alto
    short  planos;         // siempre 1
    short  bits_pixel;     // 24 para color completo
    int    compresion;     // 0 = sin compresión
    int    tam_imagen;     // tamaño de los datos (puede ser 0)
    int    ppm_x;          // píxeles por metro horizontal
    int    ppm_y;          // píxeles por metro vertical
    int    colores_tabla;  // 0 = todos
    int    colores_imp;    // 0 = todos
};
```

Después de los 54 bytes de encabezado vienen los píxeles en orden **BGR** (azul, verde, rojo), de la fila inferior a la superior. El orden invertido es un legado de la arquitectura *little-endian* de Intel.

• JPEG: compresión con pérdida

El formato **JPEG** fue estandarizado en 1992 por el *Joint Photographic Experts Group*. A diferencia de BMP, aplica compresión con pérdida: descarta información que el ojo humano percibe poco (frecuencias altas en el espacio de color YCbCr) y reduce el tamaño del archivo típicamente entre 10:1 y 20:1. Sin JPEG, las páginas web con fotografías habrían sido imprácticamente lentas en la era del módem.

Un JPEG no almacena píxeles directamente. Sus datos se organizan en **segmentos**, cada uno precedido por un marcador de dos bytes que comienza con FF:

Marcador	Hex	Significado
SOI	FF D8	Inicio del archivo (<i>Start Of Image</i>)
APP0	FF E0	Información JFIF (versión, resolución)
SOF0	FF C0	Parámetros de imagen (ancho, alto, canales)
DHT	FF C4	Tablas Huffman (para descomprimir)
SOS	FF DA	Inicio de los datos comprimidos
EOI	FF D9	Fin del archivo (<i>End Of Image</i>)

Por eso los primeros dos bytes de todo JPEG son siempre FF D8: marcan el inicio del archivo. El último marcador que encontrarás al final es siempre FF D9.

• WAV: audio sin compresión

El formato **WAV** guarda audio en formato PCM (*Pulse-Code Modulation*): muestras numéricas tomadas a intervalos regulares. Un CD de audio usa 44 100 muestras por segundo, cada una de 16 bits. Un minuto en estéreo son $44,100 \times 2 \times 2 \times 60 \approx 10$ MB.

WAV usa el contenedor **RIFF** (*Resource Interchange File Format*), una secuencia de bloques donde cada uno tiene un identificador de cuatro caracteres, un tamaño y sus datos. El inicio es siempre:

```
"RIFF" [tamaño total - 8 bytes] "WAVE"
"fmt " [16] [1=PCM] [canales] [frecuencia_Hz] [...]
"data" [tamaño de las muestras] [muestras PCM...]
```

El espacio en "fmt " es parte del identificador y no puede omitirse. Las muestras son simplemente enteros de 16 o 24 bits escritos en modo binario, uno tras otro.

• ELF: el ejecutable que ya conoces

El formato **ELF** (*Executable and Linkable Format*) es el estándar para ejecutables y bibliotecas en Linux. Cada `a.out` que has producido este semestre es un ELF. Su encabezado inicial ocupa 64 bytes en arquitecturas de 64 bits:

```
struct ELFHeader {
    unsigned char magic[4]; // 7F 45 4C 46 = "\x7FELF"
    unsigned char clase;    // 1=32 bits, 2=64 bits
    unsigned char endian;   // 1=little-endian, 2=big-endian
    unsigned char version;  // siempre 1
    // ... campos de relleno ...
    unsigned short tipo;    // 2=ejecutable, 3=librería compartida
    unsigned short maquina; // 62=x86-64
    // ...
};
```

Después del encabezado vienen las secciones: `.text` (código máquina), `.data` (variables globales inicializadas), `.bss` (variables no inicializadas) y otras que el enlazador y el cargador del sistema operativo usan para ejecutar el programa.

Los formatos binarios son, en esencia, structs escritos en disco con una estructura definida por un estándar. Las habilidades que aprendiste hoy son suficientes para leer la cabecera de un BMP, verificar los bytes mágicos de cualquier archivo, o diseñar tus propios formatos. Las bases de datos de alto rendimiento como SQLite usan exactamente este principio: páginas de tamaño fijo con una estructura bien definida.

Actividad: agenda de cumpleaños binaria

Esta actividad es la continuación directa de la sesión 21, pero ahora los datos se guardan y leen en formato binario usando el struct `Persona`.

• Formato del archivo

Cada archivo personal es un único registro binario de tipo `Persona` (`sizeof(Persona)` bytes). El nombre del archivo es tu nombre en minúsculas con extensión `.cumple` (por ejemplo, `hannah.cumple`).

• Parte A: genera tu archivo (10 min)

Escribe un programa que cree tu propio archivo `.cumple`:

1. Define una variable de tipo `Persona` y llénala con tus datos reales usando `strcpy`.
2. Ábrela en modo binario con `ofstream` y escribe el struct.
3. Cierra el archivo.
4. Abre el mismo archivo con `ifstream` en modo binario, lee el struct de vuelta y muestra los datos en pantalla para verificar que la escritura fue correcta.

Sube tu archivo `.cumple` a la carpeta de Google Drive indicada por el profesor.

• Parte B: las consultas (40 min)

Descarga todos los archivos `.cumple` de tus compañeros y colócalos en una carpeta. Luego resuelve los siguientes problemas. Para cada uno escribe un programa separado.

1. Unir todos los registros en un solo archivo binario.

Lee cada archivo `.cumple` individual y escribe todos los registros en un archivo llamado `agenda.bin`. El resultado es un arreglo de `Persona` en disco: cada registro ocupa exactamente `sizeof(Persona)` bytes, uno tras otro. Puedes apoyarte en un archivo de texto auxiliar con los nombres de los archivos individuales, uno por línea.

2. ¿Cuántas personas hay en la agenda?

Abre `agenda.bin`, mueve el stream al final con `seekg(0, ios::end)` y consulta la posición con `tellg()`. Divide entre `sizeof(Persona)`. Muestra el resultado.

3. Buscar por nombre.

Pide un nombre al usuario y recorre `agenda.bin` registro por registro hasta encontrar una coincidencia. Si existe, muestra el nombre completo y la fecha de cumpleaños.

4. Acceso aleatorio: el registro k .

Pide un número k al usuario y muestra directamente el k -ésimo registro de `agenda.bin` usando `seekg`, sin leer los registros anteriores.

Ejercicios

1. Modifica `binarios2.cpp` para guardar y leer tres valores de tipo `double` en lugar de dos `int`. Verifica con `ls -l` que el archivo resultante tiene exactamente `3 * sizeof(double)` bytes.

- Define un struct `Producto` con los campos: `char nombre[40]`, `double precio` e `int cantidad`. Escribe un programa que guarde cinco productos en un archivo binario llamado `inventario.bin` y luego los lea todos y los muestre en pantalla.
- `sizeof(Persona)` no siempre es $30 + 30 + 4 + 4 = 68$. Compila y ejecuta:

```
#include <iostream>
#include "persona.h"
using namespace std;
int main(){
    cout << sizeof(Persona) << endl;
}
```

Si el resultado es mayor que 68, el compilador está agregando **relleno** (*padding*) para alinear los campos en memoria. ¿Crees que esto afecta la escritura y lectura binaria si el escritor y el lector compilan con el mismo compilador? ¿Y si los archivos se comparten entre sistemas distintos?

- Escribe un programa que abra cualquier archivo (cuyo nombre pide al usuario), lea sus primeros 8 bytes e imprima cada byte en hexadecimal con el formato `0xNN`. Pruébalo con un `.jpg`, un `.png` y el ejecutable `a.out`. Compara los resultados con la tabla de números mágicos de la sección 1.
- Extiende el ejercicio 2 para modificar el precio de un producto dado su índice (0 a 4) *sin reescribir todo el archivo*. Usa `fstream` con los modos `fstream::in` | `fstream::out` | `fstr` y el método `seekp` para posicionarte en el registro correcto antes de llamar a `write`.

Resumen

- En **modo texto** los datos se convierten a caracteres al escribir y se parsean al leer; en **modo binario** los bytes se copian directamente entre la memoria y el disco.
- La bandera `fstream::binary` activa el modo binario al abrir el archivo.
- `write((char*)&dato, sizeof(tipo))` escribe los bytes exactos del dato; `read((char*)&dato, s` los recupera.
- El cast `(char*)&x` indica a `write/read` dónde están los bytes; `sizeof` indica cuántos copiar. Son dos informaciones distintas y ambas deben ser correctas.
- Los structs con campos de tamaño fijo (`char[n]`, `int`, `double`) se serializan directamente con `write/read`. Los `string` no, porque internamente contienen punteros.
- El acceso aleatorio a registros de tamaño fijo se logra con `seekg(k * sizeof(T))` para lectura y `seekp(k * sizeof(T))` para escritura.
- Todos los formatos de archivo populares (JPEG, PNG, WAV, ELF, BMP) son structs escritos en disco con una estructura definida por un estándar.

- Los primeros bytes de cada formato forman una **firma** o **número mágico** que identifica el tipo de archivo sin depender del nombre ni de la extensión.